SPRAWOZDANIE

Systemy Wbudowane
Piotr Bednarski
IO 5.1
Lab 9: Obsługa układów peryferyjnych. Wyświetlacz graficzny.

Polecenie numer 1:
Należało napisać taką funkcję, do której jako argumenty wywołania podamy rozmiar czcionki (SMALL/MEDIUM/LARGE), numer kolumny, numer wiersza i ilość znaków. Funkcja ta ma wyczyścić napis znajdujący się na tych współrzędnych o odpowiedniej długości znaków.

Kod programu:

```
#include <targets\AT91SAM7.h>
#include "pcf8833u8_lcd.h"

// procedura opóźnienia
__attribute__ ((section(".fast")))
void delay(int n)
{
  volatile int i;

  for(i=3000*n;i>0;i--)
  {
    __asm__("nop");
  }
}
// procedura będąca rozwiązaniem zadania
void LCDClearString(int rozmiar_czcionki, int poczatek_kolumny, int
poczatek_wiersza, int ilosc_znakow)
{
  int koniec_kolumny;
  int koniec_wiersza;
  if(rozmiar_czcionki==0)
  {
    // small
    koniec_kolumny = poczatek_kolumny + 6 * ilosc_znakow;
    koniec_wiersza = poczatek_wiersza+8;
  }
  else if(rozmiar_czcionki==1)
  {
    // medium
    koniec_kolumny = poczatek_kolumny + 8 * ilosc_znakow;
    koniec_wiersza = poczatek_wiersza+8;
  }
  else
  {
    // large
    koniec_kolumny = poczatek_kolumny + 8 * ilosc_znakow;
    koniec_wiersza = poczatek_wiersza+16;
  }

LCDClearScreen(poczatek_kolumny,koniec_kolumny,poczatek_wiersza,koniec_
wiersza); }
```

```
int main (){
  // polecenie 1
  InitLCD();
  LCDSettings();
  LCDClearScreen();
  while(1)
  {
    LCDPutStr("Piotr Bednarski",50,5,MEDIUM,BLACK,WHITE);
    delay(1000);
    LCDClearString(1,5,50,15);
    LCDPutStr("Grupa IO 5.1",80,5,LARGE,BLACK,WHITE);
    delay(1000);
    LCDClearString(2,5,80,12);
    //LCDClearString(1,5,50,15);
    delay(1000);
  }
}
```

Omówienie procedury LCDClearString():
Argumentami procedury LCDClearString są 4 liczby całkowite typu int:
- rozmiar czcionki – wartość ze zbioru {0,1,2}. Wartości te odpowiadają kolejno rozmiarom czcionki SMALL, MEDIUM oraz LARGE. Gdy podamy jakąkolwiek inną wartość spoza tego zbioru, funkcja potraktuje wielkość czcionki jako LARGE.
- początek_kolumny – współrzędna oznaczająca w której kolumnie rozpoczyna się napis do wymazania
- początek_wiersza – współrzędna oznaczająca w którym wierszu znajduje się napis do wymazania
- ilość znaków – liczba całkowita oznaczająca długość napisu do wymazania

1. Deklarujemy zmienne typu int oznaczające koniec_kolumny i koniec_wiersza. Będą to skrajne współrzędne obszaru, który zastąpimy kolorem tła.
2. W zależności od rozmiaru czcionki przekazanego w argumencie wywołania procedury obliczane są zmienne pomocnicze koniec_kolumny i koniec_wiersza. Rozmiary czcionek to odpowiednio 6x8, 8x8, 8x16.
3. Wywołujemy funkcję LCDClearScreen() podając odpowiednie parametry. Funkcja ta zdefiniowana jest w bibliotece "pcf8833u8_lcd.h". Ta funkcja zastępuje wskazany obszar kolorem białym.

W module main() testujemy zdefiniowaną procedure LCDClearString() zastępując napis oznaczający numer grupy kolorem tła (białym). Czyszczenie i pojawianie się napisu odbywa się co 1 sekundę przy użyciu wyżeju zdefiniowanej funkcji opóźnienia.

Polecenie numer 2:
Należało tak zmodyfikować funkcję LCDPutStr(), aby wyświetlała tekst na wyświetlaczu LCD na przeźroczystym tle (bez wyróżniania tekstu poprzez umieszczanie go w prostokącie o nieprzeźroczystym kolorze).

Kod programu:

```
// zalaczenie niezbednych bibliotek
#include <targets\AT91SAM7.h>
#include "pcf8833u8_lcd.h"
#include "bmp2.h"

// definicja makra TRANSPARENT na -1, czyli nieuzywana wartosc przez
zaden z kolorow
#define TRANSPARENT -1


__attribute__ ((section(".fast")))
void delay(int n) //procedura opoznienia
{
  volatile int i;

  for(i=3000*n;i>0;i--)
  {
    __asm__("nop");
  }
}

void LCDPutChar2(char c, int x, int y, int size, int fColor, int
bColor) {

  extern const unsigned char FONT6x8[97][8];
  extern const unsigned char FONT8x8[97][8];
  extern const unsigned char FONT8x16[97][16];

  int i,j;
  unsigned int nCols;
  unsigned int nRows;
  unsigned int nBytes;
  unsigned char PixelRow;
  unsigned char Mask;
  unsigned int Word0;
  unsigned int Word1;
  unsigned char *pFont;
  unsigned char *pChar;
  unsigned char *FontTable[] = {(unsigned char *)FONT6x8, (unsigned
char *)FONT8x8,
  (unsigned char *)FONT8x16};

  // get pointer to the beginning of the selected font table
  pFont = (unsigned char *)FontTable[size];

  // get the nColumns, nRows and nBytes
  nCols = *pFont;
  nRows = *(pFont + 1);
  nBytes = *(pFont + 2);
```

```
  // get pointer to the last byte of the desired character
  pChar = pFont + (nBytes * (c - 0x1F)) + nBytes - 1;

  // Row address set (command 0x2B)
  WriteSpiCommand(PASET);
  WriteSpiData(x);
  WriteSpiData(x - nRows + 1);

  // Column address set (command 0x2A)
  WriteSpiCommand(CASET);
  WriteSpiData(y);
  WriteSpiData(y + nCols - 1);

  // WRITE MEMORY
  WriteSpiCommand(RAMWR);

  // loop on each row, working backwards from the bottom to the top
  for (i = nRows - 1; i >= 0; i--) {

  // copy pixel row from font table and then decrement row
  PixelRow = *pChar--;

  // loop on each pixel in the row (left to right)
  // Note: we do two pixels each loop
  Mask = 0x80;
  for (j = 0; j < nCols; j += 2) {

  // if pixel bit set, use foreground color; else use the background
color
  // now get the pixel color for two successive pixels
if(bColor != TRANSPARENT){
if ((PixelRow & Mask) == 0) Word0 = bColor;
else  Word0 = fColor;

  Mask = Mask >> 1;

  if ((PixelRow & Mask) == 0)
  Word1 = bColor;
  else
  Word1 = fColor;
  Mask = Mask >> 1;

  WriteSpiData((Word0 >> 4) & 0xFF);
  WriteSpiData(((Word0 & 0xF) << 4) | ((Word1 >> 8) & 0xF));
  WriteSpiData(Word1 & 0xFF);

}else{
if (PixelRow & (0x80 >> j)) {
            LCDSetPixel(x - i, y + j, fColor);
        }
        if (PixelRow & (0x80 >> (j+1))) {
            LCDSetPixel(x - i, y + j + 1, fColor);
        }
}

  }
  }
  // terminate the Write Memory command
```

```
  WriteSpiCommand(NOP);
 }

int LCDPutStr2(int x, int y, int Size, int fColor, int bColor, char
*pString)
{
// loop until null-terminator is seen
    int dy = y;
    while (*pString != 0x00)
    {
// draw the character
        LCDPutChar2(*pString++, x, dy, Size, fColor, bColor);
// advance the dy position
        if (Size == SMALL)
            dy += 6;
        else if (Size == MEDIUM)
            dy += 8;
        else
            dy += 8;
// bail out if dy exceeds 131
        if (dy > 131) return dy;
    }
    return dy-y;
}
int main (){
  InitLCD();
  LCDSettings();
  LCDClearScreen();
  LCDWrite130x130bmp2(bmp2);
  LCDPutStr("TEST",30,30,MEDIUM,RED,TRANSPARENT);
}
```

Aby zmodyfikować funkcję putstr, zmodyfikujmy najpierw funkcję putchar. Gdy podany w argumencie wywołania funkcji putchar kolor jest różny od zdefiniowanego w instrukcjach preprocesora TRANSPARENT, funkcja działa niezmiennie tak, jak to miało miejsce w bibliotece "pcf8833u8_lcd.h". Gdy jednak kolor równy jest transparent, zamiast używania instrukcji WriteSpiData (które ustawiają przekazane jako argumenty wywołania funkcji wysyłającej dane wszystkie kolory pikseli tła w zaznaczonym prostokącie za pomocą PASET/CASET) używana jest funkcja LCDSetPixel(), która modyfikuje kolor pikseli jedynie tych, które zawierają krój czcionki kolejnych liter, a tło pozostawiamy bez zmian. Jest to realizowane w bloku else:

```
}else{
if (PixelRow & (0x80 >> j)) {
            LCDSetPixel(x - i, y + j, fColor);
        }
        if (PixelRow & (0x80 >> (j+1))) {
            LCDSetPixel(x - i, y + j + 1, fColor);
       }
}
```

Kolejną modyfikowaną funkcją jest funkcja putstr, ponieważ ona teraz musi korzystać z nowej funkcji putchar. Zmiana polega jedynie na innej nazwie wywoływanej funkcji:

```
LCDPutChar2(*pString++, x, dy, Size, fColor, bColor);
```

Tym sposobem uzyskujemy oczekiwany efekt zadany w poleceniu tego zadania.

W module main() wywoływana jest nowa funkcja putstr2() na tle mapy bitowej bmp w celu przetestowania działania funkcji.

Polecenie numer 3:
Animacja. Należało utworzyć animację wyświetlaną na wyświetlaczu LCD, gdzie jedną z jej klatek jest własna bitmapa przekonwertowana za pomocą BmpToArray.

Kod programu:

```
#include <targets\AT91SAM7.h>
#include "pcf8833u8_lcd.h"
#include "bmp2.h"

#define TRANSPARENT -1


__attribute__ ((section(".fast")))
void delay(int n) //procedura opoznienia
{
  volatile int i;

  for(i=3000*n;i>0;i--)
  {
    __asm__("nop");
  }
}


void LCDWrite130x130bmp2(char* bmpArr) {
// Display OFF
WriteSpiCommand(DISPLAYOFF);

          // WRITE MEMORY
WriteSpiCommand(MEMWRITE);


for(int j=0; j<25350; j++) {
  WriteSpiData(bmpArr[j]);
}


// Display On
  WriteSpiCommand(DISPLAYON);
}



int main (){
  InitLCD();
  LCDSettings();
  while(1){
    LCDClearScreen();
    LCDWrite130x130bmp2(bmp2);
    delay(1000);
    LCDClearScreen();
    LCDWrite130x130bmp();
    delay(1000);
  }
```

```
}
```

W programie definiujemy procedurę opóźnienia. Definiujemy procedurę `LCDWrite130x130bmp2()` która jako argument wywołania przyjmuje tablicę znaków typu char, którą należy wyświetlić. Funkcja działa analogicznie do tej z biblioteki "pcf8833u8_lcd.h", jedyną różnicą jest odnoszenie się do tablicy znaków przekazanej jako argument, zamiast do tablicy z bmp.h.

Utworzono własną bitmapę bmp2 za pomocą BmpToArray. Jest to bitmapa o rozmiarach 130x130, a jej zawartość umieszczona jest w pliku bmp2.h w katalogu projektu. Plik ten dołączony jest do kodu źródłowego za pomocą `#include "bmp2.h"`.

W module main wyświetlana jest prosta, dwuklatkowa animacja w nieskończonej pętli while(1). Klatki animacji zmieniają się co 1 sekundę.

Warta omówienia jest jeszcze pętla `for()` wewnątrz definicji funkcji `LCDWrite130x130bmp2()`. Pętla ta wykonuje się dokładnie 25350 razy. Skąd wzięła się akurat ta liczba?

Rozdzielczość wyświetlacza LCD wynosi 130 na 130 pikseli. W każdej iteracji wywołania instrukcji WriteSpiData przesyłamy dane do dwóch pikseli. Jednak każdy piksel składa się ze trzech składowych barw podstawowych RGB, zatem ilość wysłanych danych musimy pomnożyć razy 3. Oto obliczenia ilości iteracji pętli:

$$j_{max} = \frac{130 \cdot 130 \cdot 3}{2} = 25\,350$$

Warto dodać, że gdybyśmy zmienili rozmiar bitmapy zapisanej w pliku bmp2.h np. na większy rozmiar niż maksymalna ilość dostępnego miejsca na wyświetlaczu LCD wyświetlanie zostanie wykonane niepoprawnie a układ zachowa się w sposób nieokreślony.